AI-037: MODULE SUBSYSTEM POWER ESTIMATOR VERIFICATION AND VALIDATION

by

Kenn Du

Songli Du

Lucas Song

Riley Zhang

Sonya Zhao

April 9, 2026

Table of Contents

List of Tables.....2
List of Figures.....3
1. Overview.....4
2. Empirical Data Verification Workflow.....4
2.1 Model Verification Inputs.....5
2.2 Memory Benchmark Selection.....6
2.3 Empirical Power Measurement Workflow.....7
2.4 Generating Workload JSON From Benchmark.....9
2.5 Selecting Memory Specification JSON.....10
3 Test Descriptions.....10
3.1 Idle Power Consumption.....10
3.2 Read-only Power Consumption.....12
3.3 Read/Write Power Consumption.....14
3.4 Runtime Requirement.....16
3.5 Input Format Adherence.....17
3.6 Regression Testing.....18
4. Verification Against DRAMPower Model.....20
5. Website Verification.....23
5.1 Frontend verification.....23
5.2 Backend (API) verification.....24
5.3 Full-stack / integration verification (tests/e2e/).....24
6. Validation.....25
6.1 Python Module Validation.....25
6.2 Web Application Validation.....25
7. References.....26
8. Team Contributions.....27
Appendix.....28
1. Full Idle Power Measurement Table.....28
2. Full Read-only Power Measurement Table.....28
3. Full Read/Write Power Measurement Table.....29
4. Example of PR success:.....29

List of Tables

Table 2.3.1: Example HWiNFO Log .csv File.....8
Table 3.1.1: Truncated HWiNFO Log for Idle Scenario.....11
Table 3.2.1: Truncated HWiNFO Log for Read-only Scenario..... 13
Table 3.3.1: Truncated HWiNFO Log for Read/Write Scenario..... 15

List of Figures

Figure 2.0.1: Model Input and Output Diagram..... 4
Figure 2.1.1: Verification Workflow Diagram..... 6
Figure 2.2.1: Sample MLC Benchmark Output..... 7
Figure 2.3.1: HWiNFO Sensor Panel..... 8
Figure 2.4.1: HWiNFO Reported Maximum Bandwidth..... 9
Figure 3.1.1: Idle Scenario workload.json..... 11
Figure 3.1.2: Idle Scenario Power Consumption..... 11
Figure 3.1.3: Idle Scenario Model Outputs vs. Empirical Data..... 12
Figure 3.2.1: Read-only Scenario MLC Output..... 12
Figure 3.2.2: Read-only Scenario workload.json..... 13
Figure 3.2.3: Read-only Scenario Power Consumption..... 13
Figure 3.2.4: Read-only Scenario Model Outputs vs. Empirical Data..... 14
Figure 3.3.1: Read/Write Scenario MLC Output..... 15
Figure 3.3.2: Read/Write Scenario workload.json..... 15
Figure 3.3.3: Read/Write Scenario Power Consumption..... 15
Figure 3.3.4: Read/Write Scenario Model Outputs vs. Empirical Data..... 16
Figure 3.4.1: Runtime Measurement Source Code..... 17
Figure 3.4.2: Runtime Measurement Test Output..... 17
Figure 3.5.1: Malformed JSON Test Output..... 18
Figure 3.6.1: Example Regression Test on GitHub..... 19
Figure 4.3.1: Absolute power comparison 21
Figure 4.3.2: Percentage Difference Comparison.....……...21
Figure 4.4: Power Scaling with Data Rate 22

1. Overview

The project consists of two main components that individually require their own verification strategies. These two components are listed below:

1. Memory Subsystem Power Estimator Python code.
2. Web application interface to the Memory Subsystem Power Estimator.

For verifying the Memory Subsystem Power Estimator (hereby referred to as "the model" for brevity), we employ strategies of measuring and collecting power usage data over time on actual DRAM hardware under a variety of workloads, and comparing those measurements to the model predictions under the same input workloads. In this regard, we find that the model produces predictions that are sensible[1] when compared to our collected empirical measurements.

2. Empirical Data Verification Workflow

This workflow is used to verify the DDR5 power calculation feature of our model. First, we provide a simplified diagram of the model's inputs and outputs, as this determines what information we require in order to attempt to verify the model:

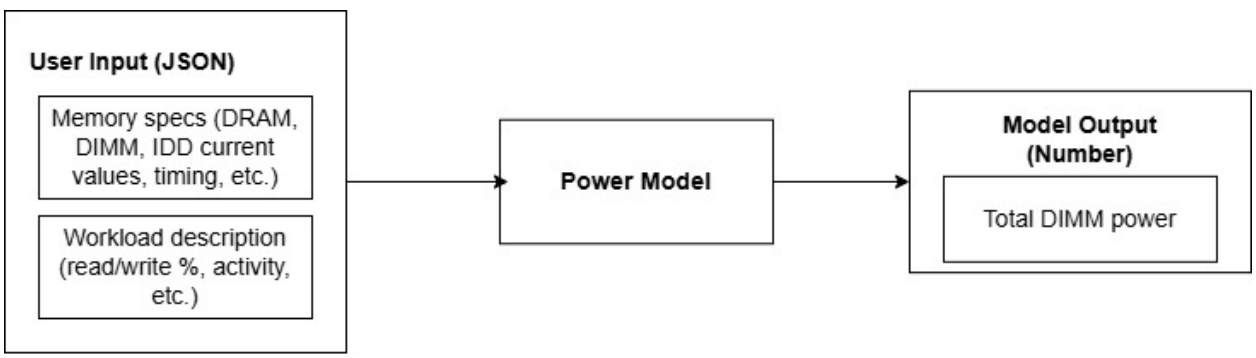


Fig. 2.0.1: A simplified view of the model inputs and outputs.

In particular, the model requires two input files:

- A memory specification JSON file that contains architectural information for the specific DIMM (memory stick) being modelled. This information is determined by the

[1] To determine "sensibility", we employ a strategy of comparing the empirical measurements to the model predictions when given a range of memory architecture specifications. This is explained in further detail in section 2.5.

manufacturer, and includes information such as the number of DRAM chips located on the DIMM, the specific IDD current values that are expected for those DRAMs, the operating voltage, and so on. Crucially, this information is related to the architecture of the memory itself, and stays constant regardless of what workload the memory is being used for.

- A workload specification JSON file that contains the distribution of different memory commands to model. This information is specific to what application the DIMM is being used for. For example, this file includes information such as the percentage of time spent reading from the DRAM, writing to the DRAM, and the percentage of time the DRAM spends idling.

With those two input files, the model produces one singular output:

- A single number (measured in W) representing the estimated total power that the DIMM will consume on average, if undergoing a workload with the ratio of memory commands specified in the workload JSON file. In other words, if a DIMM with the same memory architecture as specified in the memory specification JSON file were to be made to operate for an infinite amount of time, such that the distribution of DRAM states visited over that time matched the ratio of states as described in the workload JSON file, then the average power consumed by that DIMM is precisely the figure that our model attempts to predict. This figure is the sum of a component-wise breakdown of the power consumed by the DRAM (for more information, see the Design Document).

Understanding this allows us to specify what inputs are required for the verification of the model.

2.1 Model Verification Inputs

To verify the accuracy of the model, we assemble mappings from input files to output power figures. To do this, we employ a strategy of providing some repeatable stimulus (in the form of a memory benchmark) to a physical computer's DRAM memory, measuring the power consumed by that memory over time, and then representing that stimulus in the format of the workload

JSON file expected by our model. A high-level overview of the workflow is pictured below:

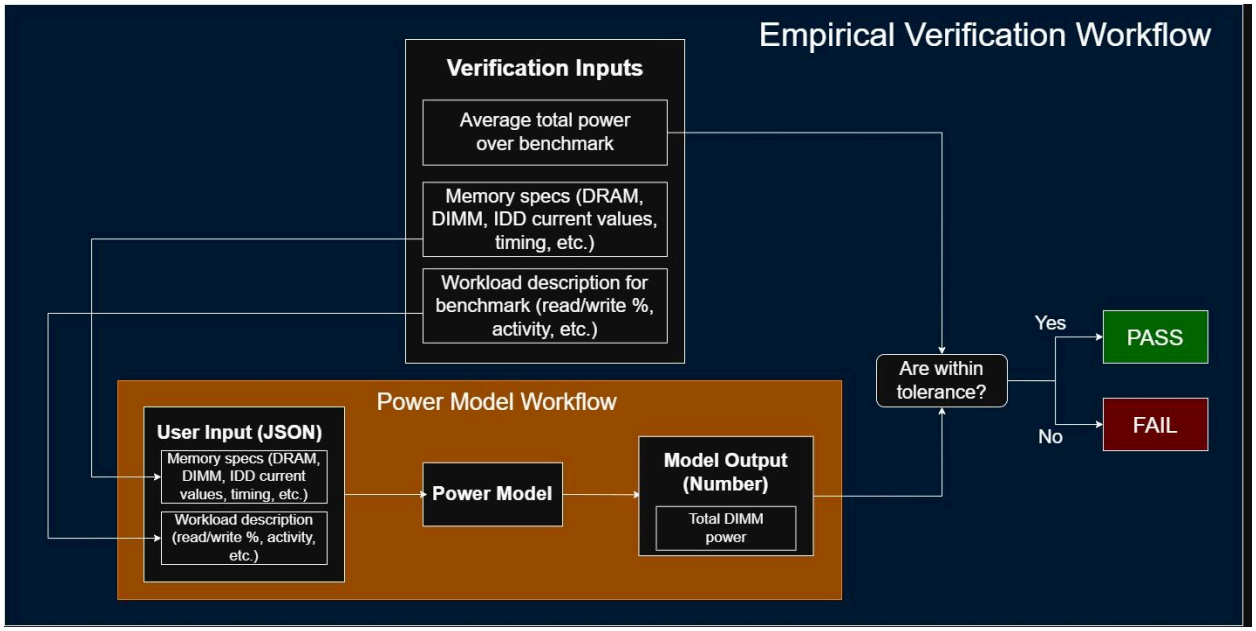


Fig. 2.1.1: Workflow for verifying model outputs with empirical data gathered from benchmarks.

As pictured, the verification workflow requires three specific inputs, alongside a memory benchmark:

- Measurements of the power consumed by the computer memory during the course of the benchmark
- Workload JSON reflecting the memory benchmark in a format compatible with the model
- Memory specification JSON modelling the DRAM of the physical computer

The following sections will describe the specific memory benchmark selected, as well as the process of generating each of the three above required inputs.

2.2 Memory Benchmark Selection

To exercise the memory on the physical computer, we utilize the Memory Latency Checker (MLC) developed by Intel [1]. The particular capabilities of this tool that are of interest to our task are its ability to generate memory traffic in a specified ratio of reads to writes (e.g. 100% reads, 0% writes, 50% reads and 50% writes, etc.), and drive that traffic to the memory on the computer. It drives this traffic in a particular manner to avoid caches and prefetchers (which

would direct some of the workload away from the system's memory in a more naive benchmark). Since our goal is to assess the power consumed by the system memory, this allows us to attain a more accurate measurement to compare with our model.

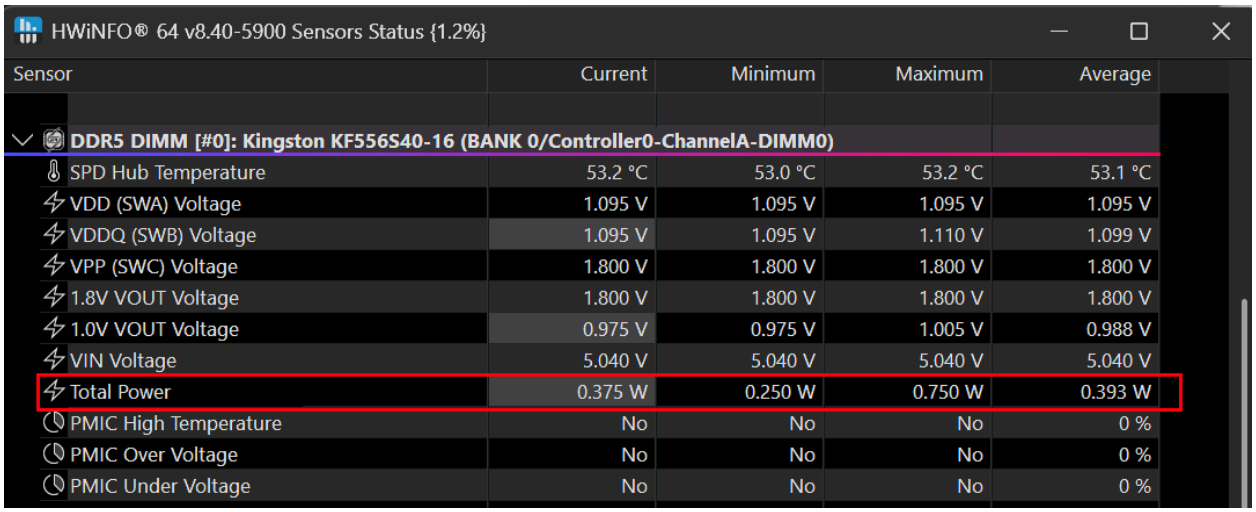
Furthermore, the tool reports the effective bandwidth and latency over the course of the benchmark. This is significant because the difference between the effective bandwidth and theoretical maximum bandwidth of the system memory allows us to estimate the percentage of time the memory spent in the read or write states, and the reported latency helps us estimate the percentage of time the memory spent idling. These allow us to represent the benchmark when generating the workload JSON file to input to our model.

```
Intel(R) Memory Latency Checker - v3.12
Command line parameters: --loaded_latency -t2 -b128m -d0 -e -r -R

Using buffer size of 128.000MiB/thread for reads and an additional 128.000MiB/thread for writes

Measuring Loaded Latencies for the system
Using all the threads from each core if Hyper-threading is enabled
Using Read-only traffic type
Inject  Latency Bandwidth
Delay   (ns)    MB/sec
==========================
 00000  223.37   62727.0
```

Fig. 2.2.1: Output after running Intel MLC benchmark; of note are the reported latency (in ns) and the bandwidth (in MB/s) at the bottom of the screen.

2.3 Empirical Power Measurement Workflow

To measure the power consumed by the DRAM on the computer over the course of the benchmark, we utilize HWiNFO, which is a free software tool for monitoring different hardware statistics on a computer [2]. In particular, HWiNFO reports, in real time, the total power consumed by each memory DIMM in the computer. This data can also be logged to a .csv file for post-processing.



HWiNFO® 64 v8.40-5900 Sensors Status {1.2%}

Sensor	Current	Minimum	Maximum	Average
DDR5 DIMM [#0]: Kingston KF556S40-16 (BANK 0/Controller0-ChannelA-DIMM0)				
SPD Hub Temperature	53.2 °C	53.0 °C	53.2 °C	53.1 °C
VDD (SWA) Voltage	1.095 V	1.095 V	1.095 V	1.095 V
VDDQ (SWB) Voltage	1.095 V	1.095 V	1.110 V	1.099 V
VPP (SWC) Voltage	1.800 V	1.800 V	1.800 V	1.800 V
1.8V VOUT Voltage	1.800 V	1.800 V	1.800 V	1.800 V
1.0V VOUT Voltage	0.975 V	0.975 V	1.005 V	0.988 V
VIN Voltage	5.040 V	5.040 V	5.040 V	5.040 V
Total Power	0.375 W	0.250 W	0.750 W	0.393 W
PMIC High Temperature	No	No	No	0 %
PMIC Over Voltage	No	No	No	0 %
PMIC Under Voltage	No	No	No	0 %

Fig. 2.3.1: The HWiNFO sensor panel for one of the memory DIMMs installed in the computer; of note is the "Total Power" sensor, outlined in red above.

Time	VDD (SWA) Voltage [V]	VDDQ (SWB) Voltage [V]	VPP (SWC) Voltage [V]	VIN Voltage [V]	Total Power [W]
09:56:36	1.095	1.095	1.8	5.04	0.75
09:56:38	1.095	1.095	1.8	5.04	2.25
09:56:40	1.095	1.095	1.8	5.04	2.25
09:56:42	1.095	1.095	1.8	5.04	2.25
09:56:44	1.095	1.095	1.8	4.97	2.375

Table 2.3.1: A few rows of the .csv log generated by HWiNFO over the course of the benchmark. Each sensor shown in Fig. 2.3.1 has its values sampled every 2 seconds and saved to this file for simple post-processing.

Therefore, the workflow for generating the power figure to compare to the model prediction is to simply start the MLC benchmark and HWiNFO logging simultaneously, extract the time series data of "Total Power [W]", and compute the average of the total power readings over the course of the benchmark.

2.4 Generating Workload JSON From Benchmark

To generate the workload JSON file to input to the model, we make the assumption that the ratio of reads to writes in the MLC benchmark will exactly be the ratio of time spent in the read and write states, scaled by the ratio of actual bandwidth to theoretical bandwidth. We compute the theoretical maximum bandwidth of the computer DRAM using the following formula:

$$BW_{max} = MT/s * bus\ width_{bytes} * \#\ channels$$

For example, a single 5600 MT/s DIMM will have a theoretical maximum bandwidth of

$$5600 * (64/8) = 44800\ MB/s$$

Which we also see reflected in the HWiNFO statistics panel for the detected hardware:

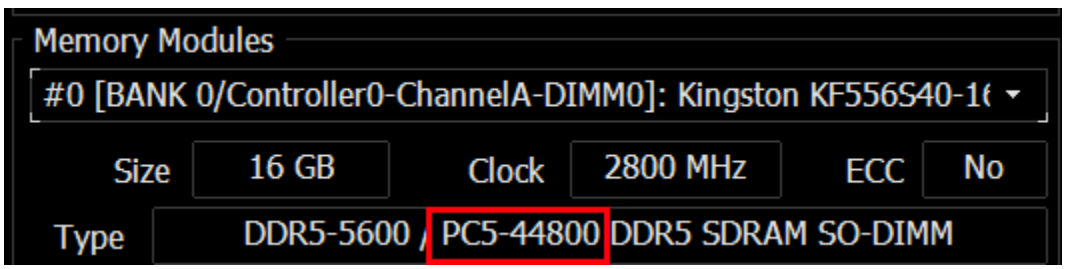


Fig. 2.4.1: Maximum theoretical bandwidth for a single DIMM on the test system. With two channels (as the system has two separate DDR5 DIMMs installed), this brings the total theoretical memory bandwidth of the system to 44800 * 2 = 89600 MB/s.

This allows us to estimate the percentage of time spent doing read or write operations. If we consider the output from MLC as shown in Fig 2.2.1, where the actual bandwidth was only 62727 MB/s, that represents approximately 70% of the theoretical maximum. Then, we simply set the workload JSON fields depending on the ratio of reads or writes in the MLC benchmark. For example, if the MLC benchmark is set to only send read operations, we would set the percentage of time spent in read to be 70%. If instead the MLC benchmark was a 1:1 ratio of reads to writes, we would set the percentage of time spent in read and write to be 35% each. For the time spent in idle, we simply keep that consistent relative to the latency of different MLC benchmarks. For example, if two MLC benchmarks report latencies of 250 ns and 500 ns respectively, then we would ensure that the workload JSON files for the run with longer latency would specify twice as much time spent in the idle state.

2.5 Selecting Memory Specification JSON

The model requires memory architecture specifications (specifically, the IDD current values) to produce an estimation, but there is a challenge in perfectly representing the physical system from which we are collecting empirical measurements, simply because the datasheets for the exact memory that we are using are not publicly available. However, we note that since we are comparing to consumer-grade hardware, in principle the variation between different memory vendors' DRAM chips and their corresponding IDD values should be small. If not, then different DRAM chips from different vendors would draw largely different amounts of power, breaking compatibility with consumer motherboards and other hardware. We observed this to be the case as well from the limited datasheets we had access to. Therefore, we determined that a reasonable way around this limitation would be to collect a range of publicly available DDR5 datasheets, then for each test case, run our model over each collected datasheet to get a set of possible predictions. Then, we determine our model provides a sensible total power estimation if the empirically measured power falls between the minimum and maximum predicted power from our model, with some tolerance (for details, see sections 3.1, 3.2, 3.3).

3 Test Descriptions

We provide descriptions of each test and the requirement(s) that each one addresses. The complete set of non-truncated measured data can be found in the appendix at the end of the report.

3.1 Idle Power Consumption

Requirement(s) Addressed: NF-01 - Physical Plausibility (power output should match real-life measurements and expectations).

To test idle power consumption, we simply do not run any MLC benchmark, and just log with HWiNFO the total power consumed over a 20 second duration (to match the MLC benchmark cases which run for 20 seconds each) which provides enough data points for us to average out the steady-state idle power consumption of the device.

```
{
  "BNK_PRE_percent": 4.8,
  "CKE_LO_PRE_percent": 0.0,
  "CKE_LO_ACT_percent": 0.0,

  "RDsch_percent": 0,
  "WRsch_percent": 0,
}
```

Fig. 3.1.1: workload.json for the idle test case. Note that both RDsch_percent and WRsch_percent are set to 0 to denote no read/write activity from the memory.

Time	VDD (SWA) Voltage [V]	VDDQ (SWB) Voltage [V]	VPP (SWC) Voltage [V]	VIN Voltage [V]	Total Power [W]
09:57:45	1.095	1.11	1.8	5.04	0.375
09:57:47	1.095	1.095	1.8	5.04	0
09:57:49	1.095	1.095	1.8	5.04	0.25
09:57:51	1.095	1.095	1.815	5.04	0.25

Table 3.1.1: Truncated view of the power consumed over time in the idle scenario.

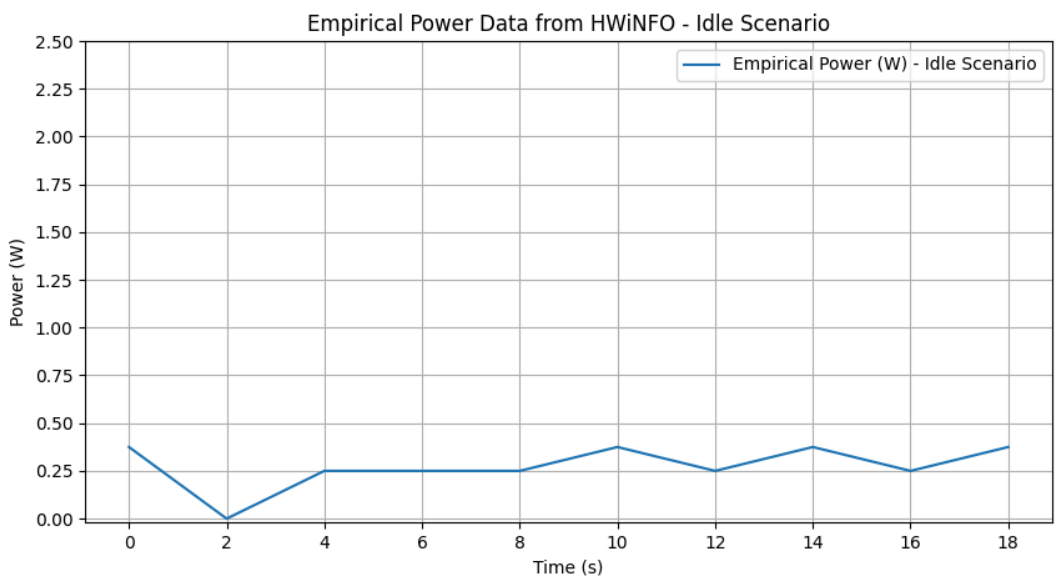


Fig 3.1.2: Plot of total power consumed in idle over time.

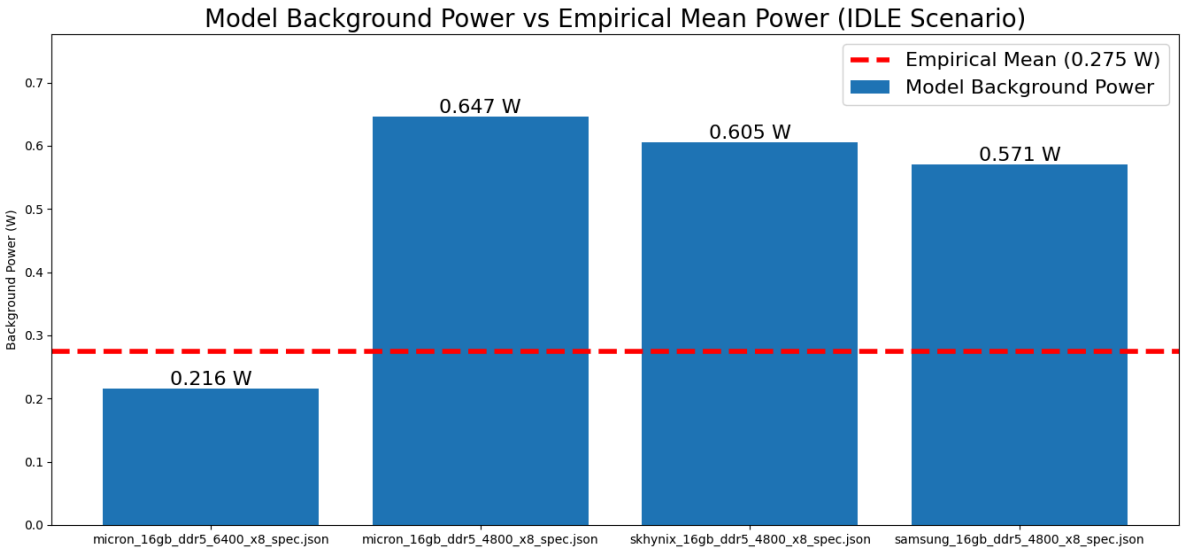


Fig. 3.1.3: The average empirical power measured (plotted as the dashed red line), and the model prediction for different memory specification JSON files. We determine this test as passing as the empirical measurement is bounded by the minimum and maximum model predictions.

3.2 Read-only Power Consumption

Requirement(s) Addressed: NF-01 - Physical Plausibility (power output should match real-life measurements and expectations).

To test the scenario where the DRAM is only doing read operations, we start the MLC benchmark using the -R flag to force purely read traffic. We then record the reported latency and bandwidth.

```
Running MLC command: sudo /root/capstone/Capstone_AI_037/core/verif/mlc_v3.12/Linux/mlc --loaded_latency -t20 -b128m -d0
 -e -r -R
Extracted Latency: 222.24 ns
Extracted Bandwidth: 63150.0 MB/s
RD Bandwidth as percentage: 70.48% (63,150.00 MB/s)
--> Use this % as input RDsch_percent in workload.json, set WRsch_percent to 0.
```

Fig. 3.2.1: Relevant MLC outputs for the read-only test case. We note that the actual bandwidth was 63150 MB/s, or about 70.48% of the theoretical maximum.

```
{
  "BNK_PRE_percent": 4.8,
  "CKE_LO_PRE_percent": 0.0,
  "CKE_LO_ACT_percent": 0.0,
  "RDsch_percent": 70.48,
  "WRsch_percent": 0,
}
```

Fig. 3.2.2: workload.json for the read-only test case. Note that both RDsch_percent is exactly the actual bandwidth observed, and WRsch_percent is 0 as we assume no writes have occurred.

Time	VDD (SWA) Voltage [V]	VDDQ (SWB) Voltage [V]	VPP (SWC) Voltage [V]	VIN Voltage [V]	Total Power [W]
09:57:10	1.08	1.095	1.8	5.04	1.625
09:57:12	1.08	1.095	1.8	5.04	1.75
09:57:14	1.08	1.095	1.815	5.04	1.75
09:57:16	1.08	1.095	1.815	5.04	1.75

Table 3.2.1: Truncated view of the power consumed over time in the read-only scenario.

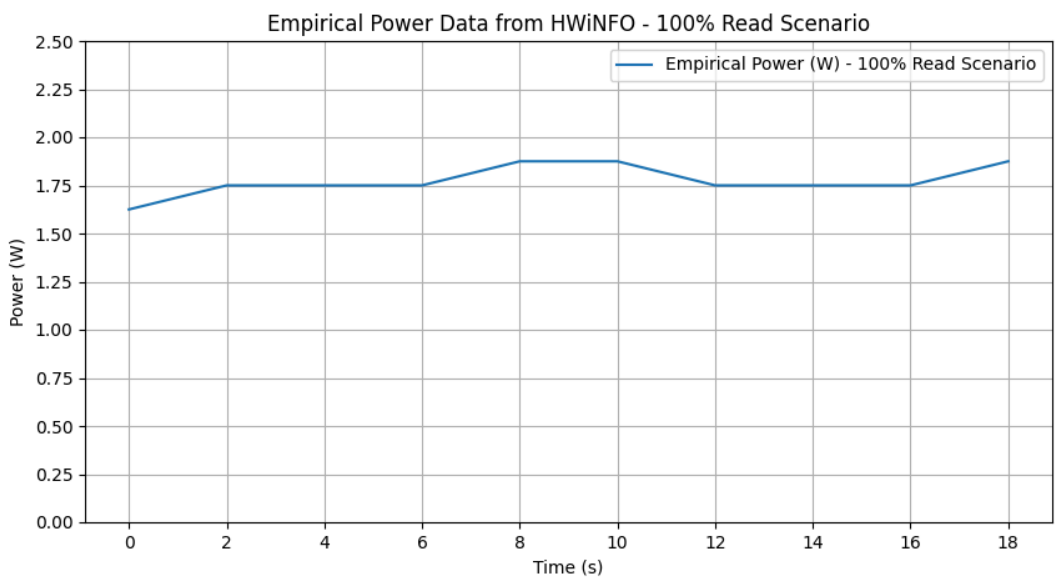


Fig. 3.2.3: Plot of total power consumed in read-only over time.

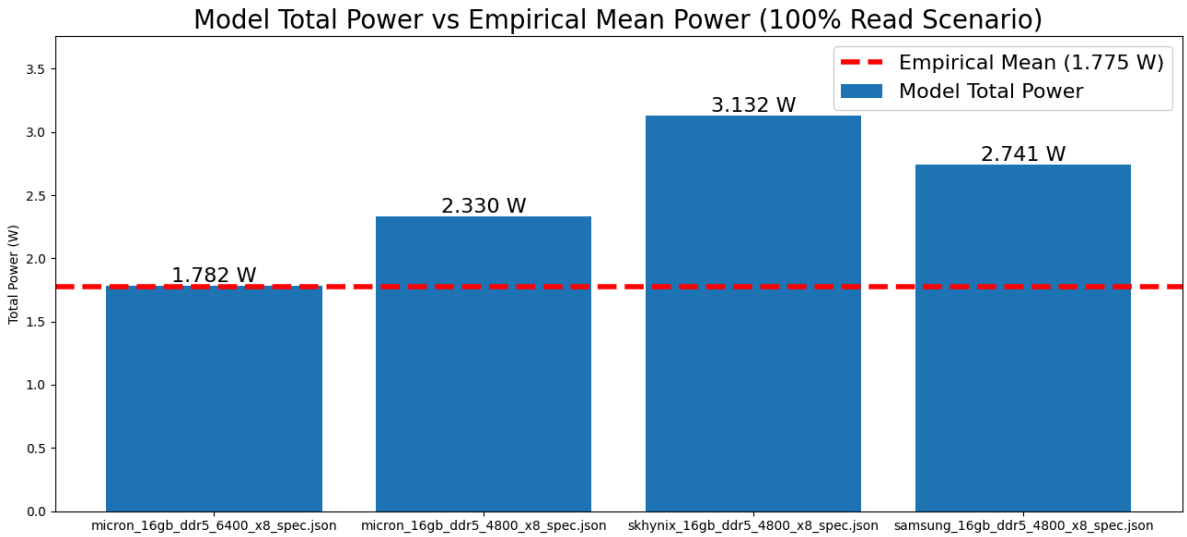


Fig. 3.2.4: Average empirical power in read-only scenario plotted against model predictions for various memory specification files. This test is deemed passing as the minimum and maximum predictions bound the empirical measurement with a margin of 5 percent.

3.3 Read/Write Power Consumption

Requirement(s) Addressed: NF-01 - Physical Plausibility (power output should match real-life measurements and expectations).

To test the scenario where there is an equal ratio of reads and writes, we start the MLC benchmark using the -W5 flag to force a 1:1 ratio of reads and writes. Again, we then record the reported latency and bandwidth.

```
Running MLC command: sudo /root/capstone/Capstone_AI_037/core/verif/mlc_v3.12/Linux/mlc --loaded_latency -t20 -b128m -d0
 -e -r -W5
Extracted Latency: 648.92 ns
Extracted Bandwidth: 57352.0 MB/s
RD/WR Bandwidth as percentage: 64.01% (57,352.00 MB/s)
--> Use this % as input RDsch_percent and/or WRsch_percent in workload.json.
```

Fig. 3.3.1: Relevant MLC outputs for read/write scenario. We note that the measured bandwidth in this case was 64.01% of the theoretical maximum, which we assume was equally split between reads and writes. Furthermore, the reported latency is longer than the read-only case, which will be reflected in the workload.json for this test case.

```
{
  "BNK_PRE_percent": 14.01,
  "CKE_LO_PRE_percent": 0.0,
  "CKE_LO_ACT_percent": 0.0,
  "RDsch_percent": 32,
  "WRsch_percent": 32,
}
```

Fig. 3.3.2: workload.json for the read/write test. We note that because the latency was longer than the read-only case, the BNK_PRE_percent here is scaled up accordingly. Furthermore, since the total bandwidth was 64.01% of the maximum, we assume an equal split between reads and writes, i.e. 32% of the time spent in read and write states.

Time	VDD (SWA) Voltage [V]	VDDQ (SWB) Voltage [V]	VPP (SWC) Voltage [V]	VIN Voltage [V]	Total Power [W]
09:56:36	1.095	1.095	1.8	5.04	0.75
09:56:38	1.095	1.095	1.8	5.04	2.25
09:56:40	1.095	1.095	1.8	5.04	2.25
09:56:42	1.095	1.095	1.8	5.04	2.25

Table 3.3.1: Truncated view of the power consumed in the read/write scenario.

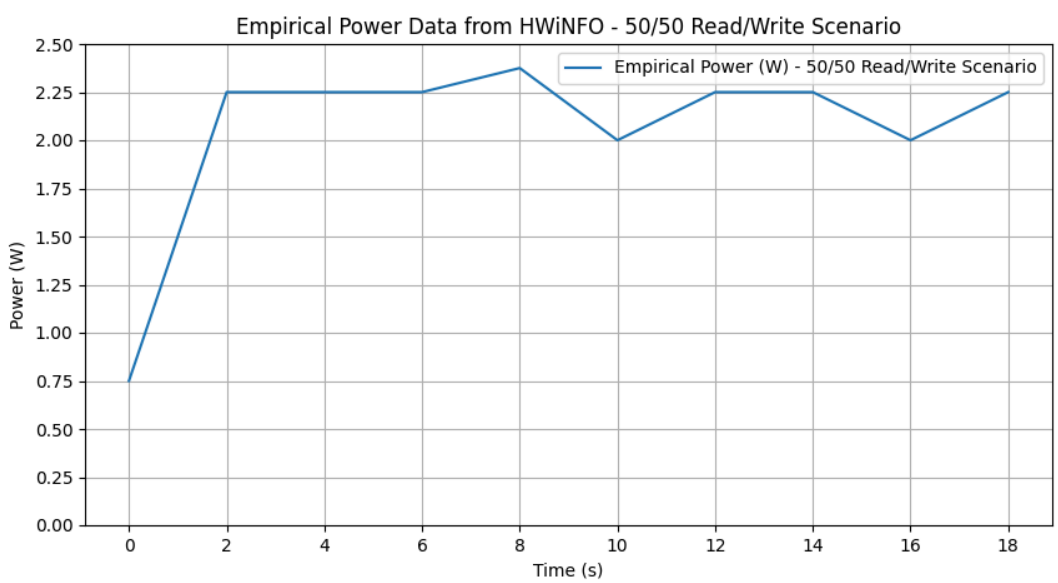


Fig. 3.3.3: Plot of power consumed in read/write scenario over time.

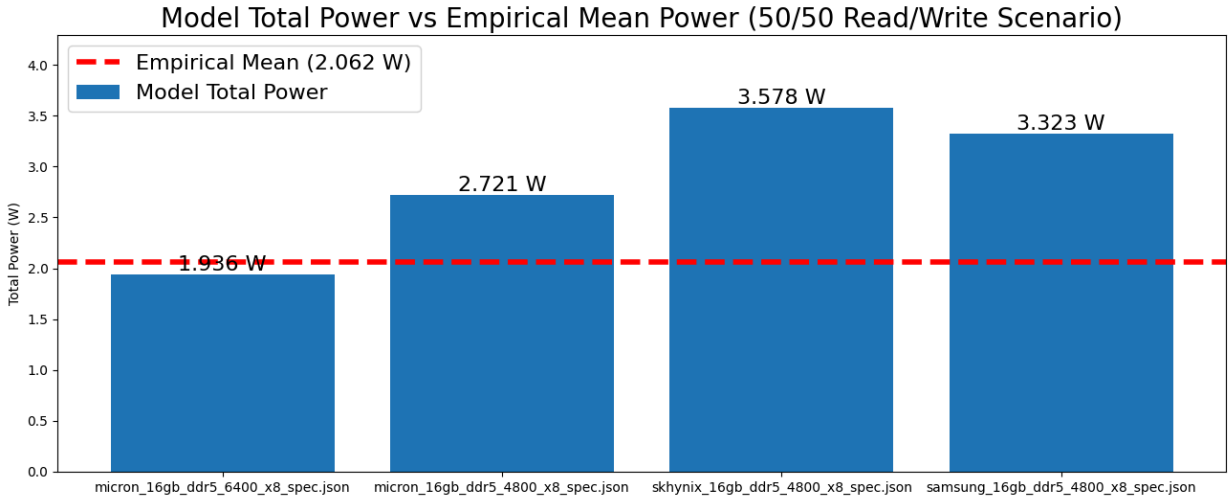


Fig. 3.3.4: Average empirical power in the read/write scenario plotted against the model predictions for different memory specification files. This test is deemed as a pass as minimum and maximum predictions bound the empirical measurement.

3.4 Runtime Requirement

Requirement(s) Addressed: NF-02 - Performance (model produces output in a time that is acceptable for users).

To measure the model performance, we simply use the Python time library to record the time elapsed to finish executing the dimm.compute_all() function, which handles all of the computation for the model's power breakdown.

```
start_time = time.time()
dimm = DIMM.load_specs(
    memspec_full,
    workload_full
)

result = dimm.compute_all()
elapsed = time.time() - start_time

print(f"  Execution time: {elapsed:.4f} seconds")
max_time = max(max_time, elapsed)
```

Fig. 3.4.1: Truncated source code for computing the execution time of the model.

```
----------------------------------------------------------------------
TEST: Runtime Performance (NF-02)
----------------------------------------------------------------------

Testing: micron_16gb_ddr5_6400_x8_spec.json
  Execution time: 0.0002 seconds
  [OK] Within time limit

Testing: samsung_16gb_ddr5_4800_x8_spec.json
  Execution time: 0.0002 seconds
  [OK] Within time limit
[PASS] NF-02: Runtime performance
   All scenarios completed in < 1s (max: 0.0002s)
```

Fig. 3.4.2: Test output for runtime performance shows execution time is much less than 1 second.

3.5 Input Format Adherence

Requirement(s) Addressed: C-03 - Input Abstraction (check that inputs are well-formatted and give useful error messages if fields are missing or malformed), C-02 - Memory Types (ensure input parameters conform to DDR5/LPDDR5 specifications), and C-04 - JEDEC Compliance (verify input parameters are defined in JEDEC specification).

Our model automatically ignores any provided parameters that are not relevant to the power estimation calculation, so it is sufficient to ensure that every required parameter is present in the input JSON.

To test this, we craft sample malformed JSON inputs and check whether the model exits gracefully (without crashing), and provides useful error messages to the user to allow them to fix the input files. In this case, a malformed JSON input can be an input that does not contain an essential parameter (thus not meeting the JEDEC/DDR5/LPDDR5 specifications), or contains an incorrect datatype for one or more parameters.

```
----------------------------------------------------------------------
TEST: Input Format Validation (C-03)
----------------------------------------------------------------------

Testing: missing_vdd
  [OK] Model rejected invalid input
  Error message: Missing required power spec field(s): vdd
  [OK] Error message is informative

Testing: invalid_timing_value
  [OK] Model rejected invalid input
  Error message: Invalid value in memtimingspec: could not convert string to float: 'invalid'
  [OK] Error message is informative
[PASS] C-03: Input format validation
   All 2 malformed inputs handled gracefully
```

Fig. 3.5.1: Test output when providing the model with inputs that are missing certain required fields. We note that the error messages helpfully specify exactly which fields were not found.

3.6 Regression Testing

Requirement(s) Addressed: F-02 - Regression Validation Tool (model should be able to store previously known accurate predictions and automatically block code changes if they result in deviation from the baseline), and NF-03 - Extensibility (ensure regression tests pass when implementing new features).

The verification Python script contains functionality to save the current model predictions as a baseline to be referred to for future model changes. If the option is provided, then the model prediction is saved as a JSON file in the project directory, which contains the expected power breakdown from the model. We then use GitHub Actions to enable automated verification on every potential code change.

Whenever a Pull Request (PR) is made to the project repository, a GitHub Actions Runner (a virtual machine) applies the code changes contained within that PR to a copy of the repository located on the Runner. It then runs the verification Python script, test_regression.py. This Python script contains all of the tests listed above. After getting the results from each of the above tests,

it compares the model predictions (with the new code changes) to the stored baseline. Then, any changes that cause significant model deviation are rejected and blocked from being merged into the main repository. The specific tests and workflows run on the virtual machine can be configured in the regression-test.yml file, allowing for the client to insert their own proprietary tests as they see fit.

```
103   Baseline file: verif/baseline/power_output_baseline.json
104   Baseline power outputs:
105     P_PRE_STBY_core: 0.053900 W
106     P_ACT_STBY_core: 0.122100 W
107     P_ACT_PRE_core: 0.008000 W
108     P_RD_core: 0.028875 W
109     P_WR_core: 0.014168 W
110     P_REF_core: 0.030145 W
111     P_VDD_core: 0.193007 W
112     P_VPP_core: 0.007007 W
113     P_total_core: 0.200014 W
114   [OK] Total power in reasonable range: 0.2000 W
115   ======================================================================
116   REGRESSION TEST SUMMARY
117   ======================================================================
118   [SUCCESS] ALL CHECKS PASSED
119   The power model:
120     * Executes successfully
121     * Produces valid numerical outputs
144     * Passes all verification requirements
145     * Matches regression baseline
```

Fig. 3.6.1. Example output of regression tests on the GitHub repository for the project.

4. Verification Against DRAMPower Model

In addition to the verification methodology described earlier, we conduct a second verification using DRAMPower[3], an open source, trace-driven DRAM power estimation tool. This verification provides confidence that our tool produces results consistent in magnitude and proportion.

4.1 Verification goal

Use DRAMPower as a reference simulator to increase confidence in our DDR5 power model by comparing power breakdowns and scaling behavior at the DIMM scope. Although the two tools do not use the same internal definition, a comparison is still valuable because it helps with exposing scope and scaling errors. The goal is not exact numericals, but reasonable agreement in breakdowns and correct scaling.

4.2 Scope

We compare DRAMPower's trace-based energy modeling, derived from command sequences from Ramulator, against our model's steady-state average power estimates derived from workload parameters. To ensure a fair comparison, both tools are aligned on critical assumptions, including geometry (device count, width, subchannels), timing parameters (tCK, tRCD, tRAS, tRP, tRFC, tREFI), voltage rails (VDD, VDDQ, VPP), and IDD/IPP current tables. Average power for DRAMPower is computed by converting total energy over the simulated time window defined by cycle count and tCK.

4.3 Deliverable 1: breakdown comparison

The first deliverable evaluates power breakdown consistency between DRAMPower and our model at the DIMM scope. The goal is to verify that both models produce reasonable per-component magnitudes and that no structural errors exist, such as missing scaling factors for device count or subchannels. Exact numerical agreement is not required or expected; instead, the comparison should confirm that both models provide consistent decomposition of power contributors and maintain realistic proportions between core and interface components.

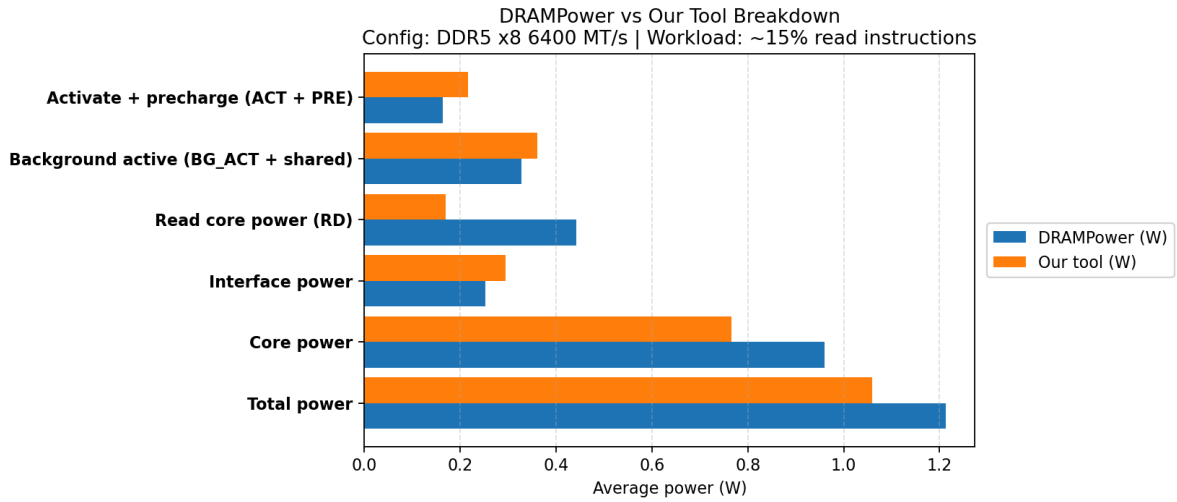


Fig 4.3.1: absolute power comparison

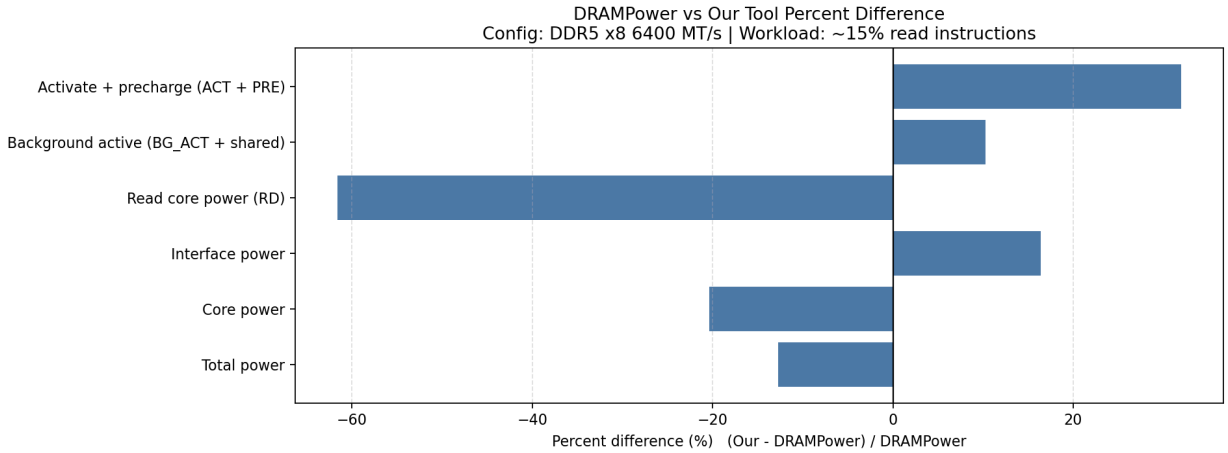


Fig 4.3.2: power percentage difference.

The first graph is the corresponding side-by-side absolute-power breakdown in watts, the second graph is a percent-difference bar chart showing how our model's predicted total, core, interface, and key core subcomponents compared DRAMPower for DDR5 x8 @ 6400 MT/s under a ~15% read workload. Overall, the results are reasonable, with total power within ~15% difference, and the high-level split between core and interface is consistent while the main area to improve is the read (RD) core power mapping, which drives most of the remaining gap.

4.4 Deliverable 2: data rate sweeping

The second deliverable validates that the model preserves correct scaling behavior through controlled parameter sweeps. The plot shows a sweep of tCK (data rate) while holding other parameters constant, with the expectation that dynamic interface power scales approximately linearly with frequency. The purpose is to ensure that the model captures scaling of physical trends.

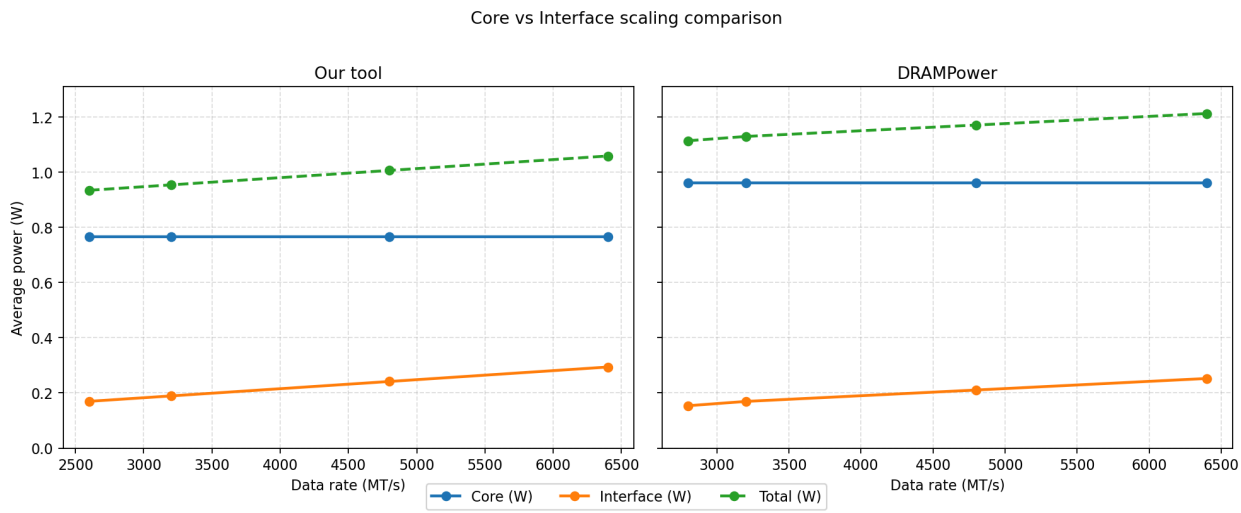


Figure 4.4: side by side data rate sweeping comparison

From the above figure, we observe that interface power increases roughly linearly with data rate because switching power follows: $P_dyn \approx C \cdot V^2 \cdot f \cdot \alpha$

where C is capacitance, V is voltage, f is frequency, and α is the activity factor. When capacitance, voltage, and activity are held constant, dynamic power is directly proportional to frequency. Therefore, higher data rates (MT/s) lead to higher interface power.

In contrast, core power remains approximately constant under a fixed-trace. The energy consumed by core operations such as READ, WRITE, ACTIVATE, and REFRESH is proportional to the time spent in each operation, which is given by (cycles × tCK). However, the

total simulation time is also proportional to (cycles × tCK). When computing average power using: P = E / T,

The tCK term cancels out. Core power depends primarily on the current differences (IDD values), rather than the clock frequency.

4.5 Conclusion:

Deliverable 1 shows that our model is in agreement with DRAMPower at the DIMM level (total power within about 15%). Deliverable 2 validates that our model reproduces the expected scaling behavior. Core power is approximately flat while interface power increases roughly linearly with data rate, making total power rise. Together, these results increase confidence that the model captures the correct trends and is consistent with a reference simulator. The next steps for verification using simulators include closing the gap from read/write power, and expanding on the scaling test to more parameters, such as read/write and DIMM geometry.

5. Website Verification

5.1 Frontend verification

Tooling: Vitest and Testing Library (npm test in frontend; also npm run lint).
What it checks: UI units (components, pages), state/helpers (e.g. builder, onboarding), and client-side API shaping—for example api.test.ts exercises normalization, mapping UI ↔ API payloads, and fallback behavior (tryApiThenLocalPower, etc.), with fetch mocked so tests stay fast and deterministic.
Pass criteria: All Vitest tests pass and lint passes; CI can upload coverage under frontend/coverage/ (artifact step is best-effort).
This validates correctness of the React/Next layer and request/response handling in code, not a live deployed URL.

5.2 Backend (API) verification

Tooling: pytest + FastAPI TestClient (no separate server process) in api/tests/.

What it checks: Health routes, and POST handlers such as /api/calculate/core, /api/calculate/interface, /api/calculate/dimm with fixture JSON; includes DDR5 and LPDDR5X-style payloads where applicable.
Pass criteria: HTTP 200 and expected keys/types in JSON (e.g. P_total_core > 0).
This verifies the HTTP API contract and that the API correctly invokes the core model, in isolation from the browser.

5.3 Full-stack / integration verification (tests/e2e/)

Despite the folder name, these are Python integration tests, not browser automation (no Playwright/Cypress usage under tests/e2e/).

test_full_stack.py
Loads sample memspec / workload via the core package, runs compute_core(), then POSTs the same data (via memspec_obj_to_api_dict / workload_obj_to_api_dict) to /api/calculate/core.
Pass criterion: API P_total_core matches direct core output within 0.1% (floating-point tolerance).

```
# Step 3: Results should be similar (allowing for small floating point differences)
core_total = core_result["P_total_core"]
api_total = api_result["P_total_core"]

# Allow 0.1% difference for floating point precision
diff = abs(core_total - api_total) / core_total
assert diff < 0.001, f"Core and API results differ by {diff*100:.2f}%"
```

test_api_integration.py

Broader API smoke tests: health, endpoint existence, valid minimal payloads.
test_vercel_config.py

Deployment contract: asserts vercel.json matches the documented unified Next + Python layout (framework, build paths, /api/(.*) → /api/index, serverless config).
Confirms the serverless entry (api/index.py) still exposes the ASGI app as expected.
The e2e workflow additionally runs npm run build in frontend before pytest, so a broken production build blocks the same pipeline that runs integration tests.

6. Validation

Due to the circumstances surrounding the client for this project, we were unable to conduct validation plans. This section will outline the strategies and rationale behind what would have been conducted for validation of the project components.

6.1 Python Module Validation

A constraint for our project is that the tool must be a self-contained python script. As a result, the command-line interface is the most likely way a potential user would be utilizing the tool. The model however requires potentially large input JSON files (the memory specification and workload), so we would have consulted with our client to determine what method of input ingress would best integrate into their existing workflows.

We decided to simply use command-line arguments pointing to local paths to specify the input files, as this would allow for the tool to be easily integrated into any shell-script workflows that a potential client may already have established, as is common in industry. If any changes were deemed necessary, then they would be implemented in our design.

6.2 Web Application Validation

While our python script is designed with the needs of more technical users in mind, our web application attempts to bridge the gap between the technical and non-technical audiences. As a result, we would survey the base of potential users of the web application and collect feedback regarding the usability of the website. We would focus on factors including:

- Does the website make clear what functions are provided (i.e., what power calculation parameters are available to be tweaked)?
- Is the user able to successfully utilize the website to accomplish their task?
- Is the meaning of each parameter on the website clear to the user?
- Are the visualizations unambiguous and convey technical information well to the user?

If any concerns about the usability of the web application were brought up, then we would work to remedy those, as improving usability would enable the technical client to utilize the web application as a visual aide in explaining power estimation results to a non-technical audience.

7. References

[1]
"Intel® Memory Latency Checker v3.12," *Intel*, 2025.
https://www.intel.com/content/www/us/en/developer/articles/tool/intelr-memory-latency-checker.html (accessed Feb. 08, 2026).
[2]
"About HWiNFO Software - Professional System Information & Diagnostics," *HWiNFO*.
https://www.hwinfo.com/about-software/
[3]
[1] DRAMPower: Open-source DRAM Power & Energy Estimation Tool
Karthik Chandrasekar, Christian Weis, Yonghui Li, Sven Goossens, Matthias Jung, Omar Naji, Benny Akesson, Norbert Wehn, and Kees Goossens
URL: http://www.drampower.info

8. Team Contributions

Section	**Major Contribution**	**Minor Contribution**	**Author**	**Reviewer**
1	KD	Riley	KD	Riley
2	KD	Riley	KD	Riley
3	KD	Riley	KD	Riley
4	Songli	Riley	Songli	Riley
5	LS		LS	
6	KD		KD	

Appendix

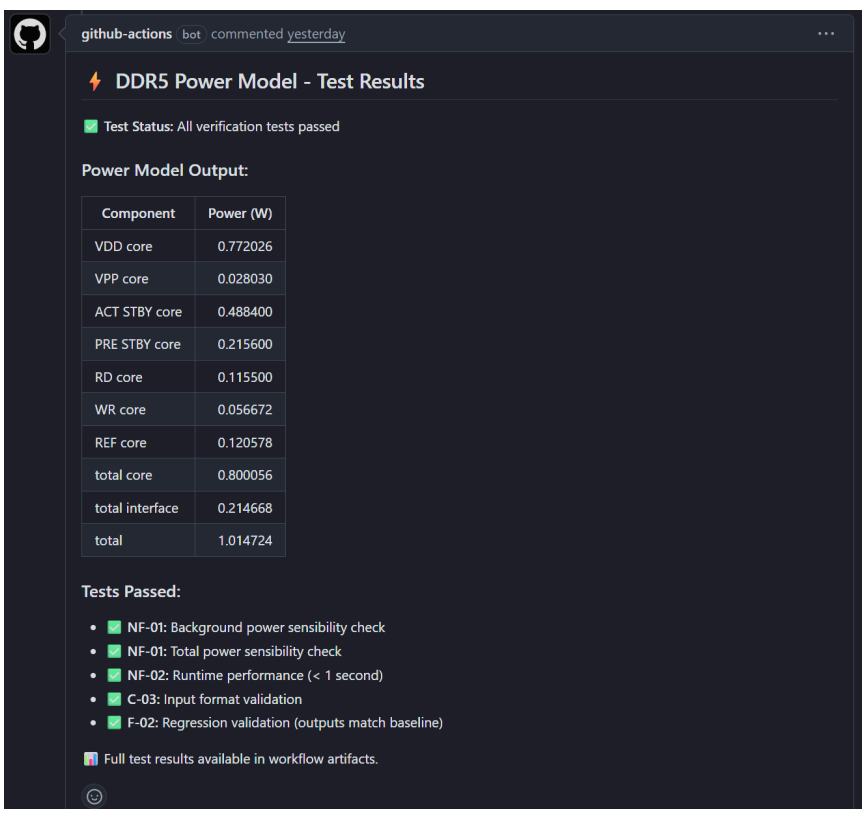
1. Full Idle Power Measurement Table

Time	VDD (SWA) Voltage [V]	VDDQ (SWB) Voltage [V]	VPP (SWC) Voltage [V]	VIN Voltage [V]	Total Power [W]
09:57:45	1.095	1.11	1.8	5.04	0.375
09:57:47	1.095	1.095	1.8	5.04	0
09:57:49	1.095	1.095	1.8	5.04	0.25
09:57:51	1.095	1.095	1.815	5.04	0.25
09:57:53	1.095	1.095	1.815	5.04	0.25
09:57:55	1.095	1.095	1.815	5.04	0.375
09:57:57	1.095	1.095	1.815	5.04	0.25
09:57:59	1.08	1.095	1.815	5.04	0.375
09:58:01	1.08	1.11	1.8	5.04	0.25
09:58:03	1.08	1.11	1.815	5.04	0.375

2. Full Read-only Power Measurement Table

Time	VDD (SWA) Voltage [V]	VDDQ (SWB) Voltage [V]	VPP (SWC) Voltage [V]	VIN Voltage [V]	Total Power [W]
09:57:10	1.08	1.095	1.8	5.04	1.625
09:57:12	1.08	1.095	1.8	5.04	1.75
09:57:14	1.08	1.095	1.815	5.04	1.75
09:57:16	1.08	1.095	1.815	5.04	1.75
09:57:19	1.08	1.095	1.815	5.04	1.875
09:57:21	1.095	1.095	1.815	5.04	1.875
09:57:23	1.08	1.095	1.815	5.04	1.75
09:57:25	1.08	1.095	1.8	5.04	1.75
09:57:27	1.08	1.095	1.815	5.04	1.75
09:57:29	1.08	1.095	1.815	5.04	1.875

3. Full Read/Write Power Measurement Table

Time	VDD (SWA) Voltage [V]	VDDQ (SWB) Voltage [V]	VPP (SWC) Voltage [V]	VIN Voltage [V]	Total Power [W]
09:56:36	1.095	1.095	1.8	5.04	0.75
09:56:38	1.095	1.095	1.8	5.04	2.25
09:56:40	1.095	1.095	1.8	5.04	2.25
09:56:42	1.095	1.095	1.8	5.04	2.25
09:56:44	1.095	1.095	1.8	4.97	2.375
09:56:46	1.08	1.095	1.8	4.97	2
09:56:48	1.08	1.11	1.8	4.97	2.25
09:56:50	1.08	1.11	1.8	4.97	2.25
09:56:52	1.08	1.11	1.8	4.97	2
09:56:54	1.08	1.11	1.8	5.04	2.25

4. Example of PR success: